

学习与支持

常见问题及解决方案

 复制页面



使用 `tool_calls` 时模型反复调用同一个工具，怎么办？

在使用工具调用 `tool_calls` 的过程中，模型可能会根据上下文连续发起多次工具调用。

如果你发现模型连续多次调用同一个工具，并且每次调用使用的 `function.name` 与 `function.arguments` 完全相同，且工具返回结果没有带来新的有效信息，可以将其视为重复工具调用。

在处理这类问题时，我们建议先排查消息布局是否正确：

1. 当 Kimi API 返回 `finish_reason=tool_calls` 时，是否已将返回的 `choice.message` 原封不动地添加到 `messages` 列表；
2. 每个 `tool_call` 是否都有一条对应的 `role=tool` 消息；
3. `role=tool` 消息中的 `tool_call_id` 是否与对应的 `tool_call.id` 完全一致；
4. 如果你使用流式输出 `stream=True`，是否已正确拼接分片返回的 `tool_calls`，尤其是 `function.arguments` 字段。

如果上述消息布局没有问题，但模型仍然重复调用同一个工具和同一组参数，可以在业务侧增加重复调用检测，并在下一轮请求的系统提示词 `system prompt` 中追加提醒。

当同一个工具和同一组参数连续重复 3 次时，可以追加：

You are repeating the exact same tool call with identical parameters. Please carefully

KIMI 开放平台 `<system-reminder>`



>

当重复调用达到 5 次时，可以追加更明确的提示，并包含工具名、重复次数和参数：

```
<system-reminder>
```

```
You have repeatedly called the same tool with identical parameters many times.
```

```
Repeated tool call detected:
```

```
- tool: {tool_name}
```

```
- repeated_times: {repeat_count}
```

```
- arguments: {tool_arguments}
```

```
The previous repeated calls did not make progress. Do not call this exact same tool
```

```
Carefully inspect the latest tool result and choose a different next action, different
```

```
</system-reminder>
```

如果同一个工具和同一组参数连续重复达到 8 次，建议再次追加上述提示。

需要注意的是，`<system-reminder>` 只是一个提示词示例，不是 Kimi API 的特殊字段。你可以将其中内容合并到下一轮请求的 `role=system` 消息中，也可以按照自己的消息管理方式写入系统提示词。为了避免误判，建议仅在“同一个工具、同一组参数、连续多次重复、工具结果没有新进展”同时成立时触发这类提示。

为什么 API 返回的结果和 Kimi 智能助手返回的结果不一致？

API 和 Kimi 智能助手使用的是同一模型，如果你发现模型输出结果不一致，可以尝试修改 System Prompt；另一方面 Kimi 智能助手提供了诸如计算器等工具，而 API 并未默认提供这些工具，需要用户自行组装；

Kimi API 是否拥有 Kimi 智能助手的“上网冲浪”功能

否。Kimi API 仅提供了大模型本身的交互功能，并不具备额外的“内容搜索”和“网页内容浏览”功能，也即是通常意义上的“联网搜索”功能。

使用 Kimi API 的联网搜索功能

KIMI 开放平台



如果你想自己通过 Kimi API 实现联网搜索功能，也可以参考我们撰写的工具调用 `tool_calls` 指南：

使用 Kimi API 完成工具调用（tool_calls）

如果你想寻求开源社区的协助，你可以参考以下开源项目：

- [search2ai](#)
- [ArchiveBox](#)

如果你想寻求由专业供应商提供的服务，有如下服务可供选择：

- [apify](#)
- [crawlbase](#)
- [jina reader](#)

Kimi API 返回的内容不完整或被截断

如果你发现 Kimi API 返回的内容不完整、被截断或长度不符合预期，你可以先检查响应体中的 `choice.finish_reason` 字段的值，如果该值为 `length`，则表明当前模型生成内容所包含的 Tokens 数量超过请求中的 `max_completion_tokens` 参数，在这种情况下，Kimi API 仅会返回 `max_completion_tokens` 个 Tokens 内容，多余的内容将会被丢弃，即上文所说“内容不完整”或“内容被截断”。

在遇到 `finish_reason=length` 时，如果你想让 Kimi 大模型接着上一次返回的内容继续输出，可以使用 Kimi API 提供的 Partial Mode，详细的文档请参考：

使用 Kimi API 的 Partial Mode

如果你想避免出现 `finish_reason=length`，我们建议你适当增大 `max_completion_tokens` 的值。推荐的最佳实践是：通过 [estimate-token-count](#) 接口计算输入内容的 Tokens 数量，然后从所选模型支持的最大上下文窗口中扣除这部分输入 Tokens。例如，`moonshot-v1-32k` 模型最大

Kimi 大模型的输出长度是多少

- 对于 `moonshot-v1-8k` 模型而言，最大输出长度是 `8*1024 - prompt_tokens`；
- 对于 `moonshot-v1-32k` 模型而言，最大输出长度是 `32*1024 - prompt_tokens`；
- 对于 `moonshot-v1-128k` 模型而言，最大输出长度是 `128*1024 - prompt_tokens`；
- 对于 `kimi-k2.6`、`kimi-k2.5`、`kimi-k2-0905-preview` 和 `kimi-k2-turbo-preview` 模型而言，最大输出长度是 `256*1024 - prompt_tokens`；

Kimi 大模型支持的汉字数量是多少？

- 对于 `moonshot-v1-8k` 模型而言，大约支持一万五千个汉字；
- 对于 `moonshot-v1-32k` 模型而言，大约支持六万个汉字；
- 对于 `moonshot-v1-128k` 模型而言，大约支持二十万个汉字；
- 对于 `kimi-k2.6`、`kimi-k2.5`、`kimi-k2-0905-preview` 和 `kimi-k2-turbo-preview` 模型而言，大约支持四十万个汉字；

注：以上均为估算值，实际情况可能有所不同。

文件抽取内容不准确、图像无法被识别

我们提供各种格式的文件上传和文件解析服务，对于文本文件，我们会提取文件中的文字内容；对于图片文件，我们会使用 OCR 识别图片中的文字；对于 PDF 文档，如果 PDF 文档中只包含图片，我们会使用 OCR 提取图片中的文字，否则仅会提取文本内容。；

注意，对于图片，我们只会使用 OCR 提取图片中的文字内容，因此如果你的图片中不包含任何文字内容，则会引起解析失败的错误。

完整的文件格式支持列表，请参考：

KIMI 开放平台
使用 `files` 接口时，

希望使用 `file_id` 引用文件内容



我们目前不支持使用文件 `file_id` 的方式引用文件内容作为上下文。

使用接口报错 `content_filter: The request was rejected because it was considered high risk`

当前请求 Kimi API 的输入或 Kimi 大模型的输出内容包含不安全或敏感内容，**注意：Kimi 大模型生成的内容也可能包含不安全或敏感内容，进而导致 `content_filter` 错误。**

出现 Connection 相关错误

如果在使用 Kimi API 的过程中，经常出现 `Connection Error`、`Connection Time Out` 等错误，请按照以下顺序检查：

1. 程序代码或使用的 SDK 是否有默认的超时设置；
2. 是否有使用任何类型的代理服务器，并检查代理服务器的网络和超时设置；

另一种可能导致 `Connection` 相关错误的场景是，未启用流式输出 `stream=True` 时，Kimi 大模型生成的 Tokens 数量过多，导致在等待 Kimi 大模型生成过程时，触发了某个中间环节网关的超时时间设置。通常，某些网关应用会通过检测是否接收到服务器端返回的 `status_code` 和 `header` 来判断当前请求是否有效，在不使用流式输出 `stream=True` 的场合，Kimi 服务端会等待 Kimi 大模型生成完毕后发送 `header`，在等待 `header` 返回时，某些网关应用会关闭等待时间过长的连接，进而产生 `Connection` 相关错误。

我们推荐启用流式输出 `stream=True` 来尽可能减少 `Connection` 相关错误。

报错信息显示 TPM、RPM 限制与我的账户 Tier 等级不匹配

如果你在使用 Kimi API 的过程遇到了 `rate_limit_reached_error` 错误，例如：

但报错信息中的 TPM 或 RPM 限制与你在后台查看的 TPM 与 RPM 并不匹配，请先排查是否正确使用了当前账户的 `api_key`；通常情况下 TPM、RPM 与预期不匹配的原因，是使用了错误的 `api_key`，例如误用了其他用户给予的 `api_key`，或个人拥有多个账号的情况下，混用了 `api_key`。

报错 `model_not_found`

请确保你在 SDK 中正确设置了 `base_url=https://api.moonshot.cn/v1`，通常情况下，`model_not_found` 错误产生的原因是，使用 OpenAI SDK 时，未设置 `base_url` 值，导致请求被发送至 OpenAI 服务器，OpenAI 返回了 `model_not_found` 错误。

Kimi 大模型出现数值计算错误

由于 Kimi 大模型生成过程的不确定性，在数值计算方面，Kimi 大模型可能会出现不同程度的计算错误，我们推荐使用工具调用 `tool_calls` 为 Kimi 大模型提供计算器功能，关于工具调用 `tool_calls`，可以参考我们撰写的工具调用 `tool_calls` 指南：

使用 Kimi API 完成工具调用（`tool_calls`）

Kimi 大模型无法回答今天的日期

Kimi 大模型无法获取像当前日期这样时效性非常强的信息，但你可以在系统提示词 `system prompt` 中为 Kimi 大模型提供这样的信息，例如：

`python` `node.js`

```
from datetime import datetime
from openai import OpenAI
```



```
client = OpenAI(
    api_key=os.environ['MOONSHOT_API_KEY'],
    base_url="https://api.moonshot.cn/v1",
)

# 我们通过 datetime 库生成了当前日期，并将其添加到系统提示词 system prompt 中
system_prompt = f"""
你是 Kimi，今天的日期是 {datetime.now().strftime('%d.%m.%Y %H:%M:%S')}
"""

completion = client.chat.completions.create(
    model="moonshot-v1-128k",
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": "今天的日期? "},
    ],
    temperature=0.3,
)

print(completion.choices[0].message.content) # 输出: 今天的日期是 2024 年 7 月 31 日。
```

在不使用 SDK 的场景下如何处理错误

在某些场合，你可能会需要自行对接 Kimi API（而不是使用 OpenAI SDK），在自行对接 Kimi API 时，你需要根据 API 返回的状态来决定后续的处理逻辑。通常而言，我们会使用 HTTP 状态码 200 表示请求成功，而使用 4xx、5xx 的状态码表示请求失败，我们会提供一个 JSON 格式的错误信息，关于请求状态具体的处理逻辑，请参考以下的代码片段：

python node.js

```
import httpx

KIMI 开放平台

header = {
    "Authorization": f"Bearer {os.environ['MOONSHOT_API_KEY']}",
}

messages = [
    {"role": "system", "content": "你是 Kimi"},
    {"role": "user", "content": "你好。"},
]

r = httpx.post("https://api.moonshot.cn/v1/chat/completions",
               headers=header,
               json={
                   "model": "moonshot-v1-128k", # <-- 如果你使用一个正确的模型，下方会
                   # "model": "moonshot-v1-129k", # <-- 如果你使用一个错误的模型名称，
                   "messages": messages,
                   "temperature": 0.3,
               })

if r.status_code == 200: # 当使用正确的模型进行请求时，会进入此分支，进行正常的处理逻辑
    completion = r.json()
    print(completion["choices"][0]["message"]["content"])
else: # 当使用错误的模型名称进行请求时，会进入此分支，在这里进行错误处理
    # 在这里，为了演示，我们仅将错误打印出来。
    # 在实际的代码逻辑中，你可能需要更多的处理逻辑，例如记录日志、中断请求或进行重试等。
    error = r.json()
    print(f"error: status={r.status_code}, type='{error['error']['type']}', message={error['error']['message']}")
```

我们的错误信息会遵循如下的格式：


```
"error": {  
  "type": "error_type",  
  "message": "error_message"  
},  
>  
}
```



具体的错误信息对照表，请参考如下章节：

错误说明

为何在提示词 prompt 相似的情况下，有的请求响应速度快，有的请求响应速度慢？

如果你遇到在相似提示词 prompt 的不同请求中，有的请求响应快（例如响应时间只有 3s），有的请求响应慢（例如响应时间长达 20s），这通常是由于 Kimi 大模型生成的 Tokens 数量不同导致的。通常而言，Kimi 大模型生成的 Tokens 数量与 Kimi API 的响应时间成正比，生成的 Tokens 数量越多，API 完整的响应时间越长。

需要注意的是，Kimi 大模型生成的 Tokens 数量只影响完整请求（指生成完最后一个 Token）的响应时间，你可以设置 `stream=True`，并观察首 Token 返回时间（首 Token 返回时间，我们简称为 TTFT — Time To First Token），通常情况下，提示词 prompt 的长度相似的场合，首 Token 响应时间不会有太大的波动。

我设置了 `max_completion_tokens=2000`，让 Kimi 输出 2000 字的内容，但 Kimi 输出的内容少于 2000 字

注：`max_tokens` 已弃用（deprecated），请使用 `max_completion_tokens`，两者含义相同。

`max_completion_tokens` 参数的含义是：调用 `/v1/chat/completions` 时，允许模型生成的最大 Tokens 数量，当模型已经生成的 Tokens 数超过设置的 `max_completion_tokens` 时，模型会停止输出下一个 Token。

1. 帮助调用方确定该使用哪个模型（例如，当 `prompt_tokens + max_completion_tokens <= 8 * 1024` 时，可以选择 `moonshot-v1-8k` 模型）；

2. 防止在某些意外的场合，Kimi 模型输出了过多不符合预期的内容，进而导致额外的费用消耗（例如，Kimi 模型重复输出空白字符）；

`max_completion_tokens` 并不能指示 Kimi 大模型输出多少 Tokens，换句话说，`max_completion_tokens` 不会作为提示词 `prompt` 的一部分输入 Kimi 大模型，如果你想让模型输出特定字数的内容，可以参考以下通用的解决办法：

- 对于要求输出内容字数在 1000 字以内的场合：
 1. 在提示词 `prompt` 中向 Kimi 大模型明确输出的字数；
 2. 通过人工或程序手段检测输出的字数是否符合预期，如果不符合预期，通过在第二轮对话中向 Kimi 大模型指示“字数多了”或“字数少了”，让 Kimi 大模型输出新一轮的内容。
- 对于要求输出内容字数在 1000 字以上甚至更多时：
 1. 尝试将预期输出的内容按结构或章节切割成若干部分，并制成模板，并使用占位符标记想要 Kimi 大模型输出内容的位置；
 2. 让 Kimi 大模型按照模板，逐个填充每个模板的占位符部分，最终拼装成完整的长文文本。

我在一分钟内只请求了一次，但却触发了 `Your account reached max request` 错误

通常，OpenAI 提供的 SDK 包含了重试机制：

Certain errors are automatically retried 2 times by default, with a short exponential backoff. Connection errors (for example, due to a network connectivity problem), 408 Request Timeout, 409 Conflict, 429 Rate Limit, and ≥ 500 Internal errors are all retried by default.

这种重试机制在遇到错误时，会默认重试 2 次（总计 3 次请求），通常来说，对于网络状况不稳定或者其他可能导致请求发生错误的场合，使用 OpenAI SDK 会将一个请求放大至 2 到 3 次请求，这些

注：对于使用 OpenAI SDK 且账户等级为 `tier0` 的用户而言，由于存在默认的重试机制，一次错误的请求就会消耗完所有的 RPM 额度。

为了便于传输，我使用 `base64` 编码我的文本内容

请不要这样做，使用 `base64` 编码你的文件会导致产生巨量的 Tokens 消耗。如果你的文件类型是我们 `/v1/files` 文件接口支持的格式，使用文件接口上传并抽取文件内容即可。

对于二进制或其他格式编码的文件，Kimi 大模型暂时无法解析内容，请不要添加到上下文中。

为什么我在 `platform.kimi.ai` 平台申请的 key，不能用在 `platform.kimi.com` 平台？

Kimi 开放平台官方提供两个平台，中国境内建议使用 `platform.kimi.com` 平台，境外建议使用 `platform.kimi.ai` 平台。两个平台的账户和 key 完全独立，不能混用。

如果用错会出现 `401 invalid_authentication_error` 的报错，收到 401 报错请先检查是否平台的 key 使用错误。

- 国内开放平台 base_url: <https://api.moonshot.cn/v1>
- 境外开放平台 base_url: <https://api.moonshot.ai/v1>

此页面对您有帮助吗？

 是

 否

🎉 Kimi K2.6 模型已正式发布，长程代码编写能力更强更稳。



KIMI

开放平台



>